UNIVERSIDADE FEDERAL DO RIO GRANDE — FURG
CENTRO DE CIÊNCIAS COMPUTACIONAIS — C3

DISCIPLINA:	Compiladores
DOCENTE:	Prof. Dr. Gerson Alberto Leiria Nunes
SEMESTRE ACADÊMICO:	2026/2
ATIVIDADE AVALIATIVA:	Trabalho Prático 1 — Front-End Completo com PLY
VALOR DA ATIVIDADE:	30% da Nota Bimestral (Grupos de até 2 integrantes)

ESPECIFICAÇÃO TÉCNICA E DIRETRIZES DO TRABALHO PRÁTICO 1 (TP1)
Front-End de Compilador Pascal-S com Python PLY

1. OBJETIVOS DO TRABALHO

Este trabalho prático tem como propósito central a aplicação rigorosa e integrada dos conceitos teóricos de Análise Léxica, Análise Sintática, amplamente estudados nas Aulas de 1 a 6 desta disciplina. Os discentes deverão projetar e implementar o *front-end* completo de um compilador voltado a um subconjunto restrito da linguagem de programação Pascal, denominado **Pascal-S**. A atividade exigirá o emprego estruturado da biblioteca PLY (Python Lex-Yacc), forçando os grupos a dominarem as nuances de engenharia de *software* aplicadas à construção de compiladores industriais e didáticos. Especificamente, os objetivos pedagógicos e técnicos desta atividade compreendem:

- **Implementação de análise léxica sob demanda:** Modelar a microssintaxe da linguagem Pascal através de expressões regulares, assegurando que a fita de entrada seja processada de maneira cooperativa e preguiçosa (*lazy*) via chamadas iterativas a um método equivalente a getNextToken(), impedindo leituras totais e estáticas de arquivos em memória, conforme discutido na Aula 2.
- **Desenvolvimento de tabela de símbolos como repositório central:** Estruturar um repositório centralizado baseado em tabela *hash* ou pilha de ambientes para gerenciar escopos aninhados e registrar os atributos essenciais de cada identificador (tipo, escopo, deslocamento, linha de declaração e ponteiros na tabela), conforme os preceitos de consistência estática.
- **Construção de parser LALR(1) não ambíguo:** Elaborar uma especificação sintática em Gramática Livre de Contexto (GLC) na notação EBNF, implementando-a no gerador de analisador sintático yacc.py do PLY, tratando explicitamente conflitos de *shift/reduce* e *reduce/reduce* com precedências e associatividades bem-definidas.
- **Mecanismo de Recuperação de Erros Sintáticos em Modo Pânico:** Desenvolver uma rotina avançada no callback p_error(p) que permita ao compilador sincronizar o fluxo de análise ao encontrar o token ponto e vírgula (;), descartando *tokens* inadequados até encontrar o próximo caractere estável e garantindo a listagem de múltiplos erros de compilação em um único passo.
- **Validação Semântica de Tipagem Forte:** Aplicar regras de verificação estática estritas na árvore de derivação e na tabela de símbolos, onde qualquer tentativa de mistura ou operação direta entre tipos numéricos inteiros (*integer*) e reais (*real*) resulte em um **erro de tipo incompatível**, abortando a compilação de forma coordenada.

2. ESPECIFICAÇÃO DA LINGUAGEM PASCAL-S

A linguagem de programação **Pascal-S** é um subconjunto imperativo estruturado e fortemente tipado da linguagem Pascal clássica. As regras a seguir definem os aspectos léxicos (microssintaxe) e sintáticos (macro-sintaxe) formais que o compilador deve reconhecer.

2.1 Microssintaxe (Tokens e Padrões Léxicos)

O analisador léxico (*scanner*) deverá quebrar o código fonte de entrada em lexemas bem-definidos, mapeando-os nos seguintes ***tokens***. Note que o *scanner* deve obrigatoriamente descartar comentários de linha (iniciados com "//") e comentários de bloco delimitados por chaves ("{ ... }"), conforme o padrão clássico do Pascal.

Categoria / Token	Lexemas Exemplo	Expressão Regular / Padrão Informal
Palavras Reservadas	**program, var, integer, real, begin, end, if, then, else, while, do, writeln, not, and, or**	Correspondência textual exata, case-insensitive.
Identificadores (ID)	**total, soma, x2, valor_calculado**	[a-zA-Z_][a-zA-Z0-9_]* (Não se pode iniciar por dígito; palavras reservadas devem ser filtradas por tabela *hash*).
Constantes Inteiras	**10, 0, 325, 9999**	Sequência de um ou mais dígitos decimais: **\d+**
Constantes Reais	**3.1415, 0.5, 120.0**	Dígitos seguidos por ponto e dígitos obrigatórios: **\d+\.\d+**
Operador Atribuição	**:=**	Combinação literal exata de dois-pontos e igual **:=**
Op. Relacionais	**=, <>, <, <=, >, >=**	Símbolos isolados ou combinados mapeados na categoria **relop (LT, LE, EQ, NE, GT, GE).**

2.2 Gramática Livre de Contexto (GLC)

A estrutura do código fonte **Pascal-S** deve obedecer estritamente à gramática não ambígua a seguir, expressa na notação EBNF (*Extended Backus-Naur Form*). Regras sintáticas com recursão à esquerda devem ser priorizadas para otimizar a pilha do analisador LALR(1) fornecido pelo gerador sintático.

```
1.  program           ::= "program" identifier ";" decl_list block "."
2.  decl_list         ::= [ "var" decl_group_list ]
3.  decl_group_list   ::= decl_group { decl_group }
4.  decl_group        ::= id_list ":" type ";"
5.  id_list           ::= identifier { "," identifier }
6.  type              ::= "integer" | "real"
7.  block             ::= "begin" stmt_list "end"
8.  stmt_list         ::= stmt { ";" stmt }
9.  stmt              ::= assign_stmt | if_stmt | while_stmt | write_stmt | block |
empty
10. empty             ::=
11. assign_stmt       ::= identifier ":=" expr
12. if_stmt           ::= "if" expr "then" stmt [ "else" stmt ]
13. while_stmt        ::= "while" expr "do" stmt
14. write_stmt        ::= "writeln" "(" expr_list ")"
15. expr_list         ::= expr { "," expr }
16. expr              ::= simple_expr [ relop simple_expr ]
17. simple_expr       ::= term { addop term }
18. term              ::= factor { mulop factor }
19. factor            ::= identifier | number | "(" expr ")" | "not" factor
20. relop             ::= "=" | "<>" | "<" | "<=" | ">" | ">="
21. addop             ::= "+" | "-" | "or"
22. mulop             ::= "*" | "/" | "and"
```

3. DIRETRIZES DE IMPLEMENTAÇÃO EM PYTHON (PLY)

A implementação do compilador deverá se dividir em módulos bem isolados, utilizando de maneira correta e modular a arquitetura da biblioteca PLY, integrando de forma coesa a análise léxica, sintática e a tabela de símbolos.

3.1 Módulo Léxico (lex.py) e Interação sob Demanda

O analisador léxico deve ser projetado de acordo com as especificações formais de autômatos para *tokens*. Dentre as exigências rigorosas deste trabalho, destaca-se que a análise léxica não pode ocorrer de forma estática ou estritamente em lote. Em vez de ler todo o arquivo de entrada e passar um vetor de *tokens* estáticos, o analisador deve operar de forma sob demanda, onde o analisador sintático invoca preguiçosamente o *callback* de *tokenização*.

No PLY, este comportamento é alcançado pela integração natural com o yacc.py. Os alunos devem expor e garantir que a leitura do arquivo utiliza *buffers* eficientes e que a função *parser* se comunica de forma bidirecional e preguiçosa. Cada chamada do *parser* que consome o *lookahead* realiza o *getNextToken*() internamente através da função *token()* do objeto *lexer*, casando dinamicamente o maior prefixo viável (*longest match*) e atualizando o contador físico de linhas do programa para reporte exato de erros de compilação.

3.2 Módulo de Tabela de Símbolos e Gestão de Atributos

A Tabela de Símbolos (TS) representa o "repositório central de atributos" do compilador, sendo a estrutura responsável pela consistência de escopos léxicos e verificação estática. As seguintes regras técnicas são obrigatórias na modelagem da TS:

- **Representação por Ponteiros:** O valor do atributo associado a um token do tipo ID (identificador) inserido ou consultado no fluxo léxico deve ser obrigatoriamente um ponteiro (ou referência em Python) que aponte diretamente para o respectivo nó ou entrada na Tabela de Símbolos, na qual todas as propriedades complexas da variável estão centralizadas (nome, escopo, tipo básico e linha física de declaração).
- **Gerenciamento de Ambientes por Escopos:** A estrutura de dados interna da TS deve dar suporte à representação de ambientes hierárquicos aninhados (utilizando tabelas *hash* encadeadas através de referências "*prev*" ou uma pilha de ambientes). Toda vez que o analisador entrar em um novo bloco de instruções do **Pascal-S** (por exemplo, dentro de um escopo local futuro ou corpo de subprograma), deve-se instanciar uma tabela filha, cujo ***look-up*** pesquisa incrementalmente do escopo atual em direção ao pai, garantindo a resolução estática correta.
- **Prevenção de Duplicidades e Omissões:** O compilador deve disparar mensagens de erro se detectar que uma variável está sendo declarada novamente sob o mesmo escopo lógico (**redeclaração Ilegal**), ou se uma variável for utilizada em uma expressão sem que tenha sido previamente registrada em seu devido escopo léxico de declaração (**variável não declarada**).

3.3 Módulo Sintático e Verificador Semântico (yacc.py)

O analisador sintático (yacc.py) utilizará geradores LALR(1) para processar o fluxo de *tokens* sob demanda. O verificador semântico deve atuar em conjunto com o *parser* (através de regras de tradução ou avaliação dirigida por sintaxe).

Uma exigência central e rigorosa deste projeto avaliado reside no suporte restrito à tipagem estática forte da linguagem Pascal-S. **Qualquer forma de coerção ou conversão implícita entre os tipos primitivos *integer* e *real* é expressamente proibida**. Portanto, operações aritméticas binárias (+, -, *, /) ou operadores relacionais envolvendo operandos de tipos distintos (por exemplo, somar uma variável **integer** a um valor **real**) devem disparar **erro semântico de incompatibilidade de tipos** de forma imediata. Da mesma forma, atribuições em que o tipo da variável à esquerda difere do tipo resultante da expressão à direita (**atribuir uma constante *real* a um *integer***) devem ser invalidadas semanticamente de imediato.

3.4 Tratamento e Recuperação de Erros em Modo Pânico

O compilador não deve interromper sua execução imediatamente após se deparar com a primeira incoerência sintática no código fonte. Os grupos deverão projetar e implementar um algoritmo de recuperação em **modo pânico** de forma robusta no yacc.py, utilizando o ***callback p_error(p)*** do PLY.

Ao capturar um erro sintático (topo da pilha sem transição válida com *lookahead*), a rotina de erro deve relatar uma mensagem precisa ao usuário, indicando a **linha física, a coluna e o *token incorreto*** detectado. Em seguida, o analisador sintático deve adotar o ponto e vírgula (";") como token de sincronização definitivo. O *parser* deverá descartar sequencialmente todos os *tokens* subsequentes da entrada até encontrar o próximo ponto e vírgula, momento no qual a pilha de análise sintática é reajustada de forma coordenada para que o compilador retome a análise de forma normal. **Esse mecanismo permite que múltiplos erros sintáticos sejam listados com precisão em uma única execução.**

3.5 Esqueleto de Inicialização (*Boilerplate*)

Para dar início ao trabalho, recomenda-se a utilização do esqueleto de código estruturado abaixo. Os discentes devem expandir e completar este esqueleto, adicionando as estruturas de escopo da Tabela de Símbolos, regras semânticas e o parser detalhado.

```
import ply.lex as lex
import ply.yacc as yacc

# 1. DEFINIÇÃO DE TOKENS E PALAVRAS RESERVADAS (revisem pois certamente esqueci várias)
reserved = {
    'program': 'PROGRAM', 'var': 'VAR', 'integer': 'INTEGER', 'real': 'REAL',
    'begin': 'BEGIN', 'end': 'END', 'if': 'IF', 'then': 'THEN', 'else': 'ELSE',
    'while': 'WHILE', 'do': 'DO', 'writeln': 'WRITELN', 'not': 'NOT',
    'and': 'AND', 'or': 'OR'
}

tokens = [
    'ID', 'NUM_INT', 'NUM_REAL', 'ASSIGN', 'COLON', 'SEMICOLON', 'COMMA', 'DOT',
    'LPAREN', 'RPAREN', 'PLUS', 'MINUS', 'TIMES', 'DIVIDE', 'EQ', 'NE', 'LT', 'LE', 'GT', 'GE'
] + list(reserved.values())

# Expressões Regulares de Símbolos
t_ASSIGN = r':='
t_COLON = r':'
t_SEMICOLON = r';'
t_COMMA = r','
t_DOT = r'\.'
t_LPAREN = r'\('
t_RPAREN = r'\)'
t_PLUS = r'\+'
t_MINUS = r'-'
t_TIMES = r'\*'
t_DIVIDE = r'/'
t_EQ = r'='
t_NE = r'<>'
t_LT = r'<'
t_LE = r'<='
t_GT = r'>'
t_GE = r'>='

t_ignore = ' \t'

def t_NUM_REAL(t):
    r'\d+\.\d+'
    t.value = float(t.value)
    return t

def t_NUM_INT(t):
    r'\d+'
    t.value = int(t.value)
    return t
```

```
def t_ID(t):
    r'[a-zA-Z_][a-zA-Z0-9_]*'
    # Case insensitivity para palavras reservadas
    t.type = reserved.get(t.value.lower(), 'ID')
    return t

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

def t_error(t):
    # Reporte de Erro Léxico Real com linha física e caractere inválido
    print(f"Erro Léxico na linha {t.lexer.lineno}: Caractere ilegal '{t.value[0]}'
detectado.")
    t.lexer.skip(1)

lexer = lex.lex()

# 2. ESTRUTURA DA TABELA DE SÍMBOLOS (REPOSITÓRIO DE ATRIBUTOS)
class Symbol:
    def __init__(self, name, var_type, lineno):
        self.name = name
        self.type = var_type
        self.lineno = lineno

class SymbolTable:
    def __init__(self, parent=None):
        self.symbols = {}
        self.parent = parent

    def insert(self, name, symbol):
        if name in self.symbols:
            return False # Erro de Dupla Declaração no escopo atual
        self.symbols[name] = symbol
        return True

    def lookup(self, name):
        # Busca incremental estática
        table = self
        while table is not None:
            if name in table.symbols:
                return table.symbols[name]
            table = table.parent
            # Retorna ponteiro/referência direta
        return None

# fiquem à vontade para melhorar, acrescentar e mudar tudo que se faz necessário.
```

4. CASOS DE TESTE E PROTOCOLO DE VALIDAÇÃO

Todos os compiladores submetidos pelos grupos de discentes serão confrontados com um protocolo rigoroso de validação da coordenação da disciplina. Recomenda-se enfaticamente testar exaustivamente o *front-end* nos três cenários obrigatórios detalhados a seguir.

Cenário 1: Sucesso Sintático e Semântico Completo (Tipos Homogêneos)

Neste cenário, o código fonte **Pascal-S** de entrada deve ser perfeitamente livre de falhas de digitação, sintaxe ou tipagem. As declarações de variáveis ocorrem homogeneamente no início do código, seguidas por blocos estruturados e laços bem-definidos.

```
program SucessoSemantico;
var
    cont, limite : integer;
    taxa, acumulador : real;
begin
    cont := 0;
    limite := 100;
    taxa := 1.25;
    acumulador := 0.0;
    while cont < limite do
    begin
        cont := cont + 1;
        acumulador := acumulador + taxa
    end;
    writeln(cont, acumulador)
end.
```

Cenário 2: Falha Léxica por Caractere Ilegal

O compilador deve reportar e recuperar erros de caracteres estranhos à gramática, reportando exatamente a linha e coluna da ocorrência durante o ***getNextToken()*** e permitindo ao parser recuperar-se até o fim do bloco.

```
program TesteErroLexico;
var
    cont : integer;
begin
    cont := 10 @ 5; // Erro léxico: '@' é ilegal na linguagem Pascal-S
    writeln(cont)
end.
```

Cenário 3: Falha de Tipagem Forte (Semântica)

Neste cenário de inconsistência estática, o programa possui sintaxe válida, mas viola a tipagem forte do **Pascal-S.** A atribuição de uma expressão *real* (**acumulador + taxa**) a uma variável inteira (**cont**) deve disparar um aborto imediato na verificação estática.

```
program IncompatibilidadeDeTipos;
var
    cont : integer;
    acumulador, taxa : real;
begin
    acumulador := 5.5;
    taxa := 1.1;
    cont := acumulador + taxa; // Erro semântico impeditivo: float atribuído a integer
    writeln(cont)
end.

# Façam vários código com erros e criem uma bateria de testes complexos, pois o professor vai
testar a fim de procurar erros nos analisadores dos alunos
```

5. CRITÉRIOS DE AVALIAÇÃO (BAREMA) E ENTREGÁVEIS

A avaliação deste Trabalho Prático (TP1) considerará de forma extremamente rigorosa o esforço de engenharia, a fidelidade do compilador frente às especificações formais de EBNF e a clareza metodológica do relatório técnico.

Item de Avaliação	Peso	Critérios de Rigor e Qualidade
Analisador Léxico (Scanner)	25%	Fidelidade às expressões regulares dos ***tokens***, descarte preciso de espaços/comentários de múltiplos formatos, manipulação física de linhas no ***lexer*** e suporte rigoroso à tokenização preguiçosa sob demanda (***getNextToken***).
Tabela de Símbolos & Tipos	20%	Estrutura de ambientes para escopo estático de variáveis, uso obrigatório de ponteiros para registros nas ocorrências de ***tokens ID***, detecção precoce de variáveis não declaradas ou dupla declaração.
Analisador Sintático (Parser)	35%	Implementação estrita das produções da gramática livre de contexto (GLC) do ***Pascal-S***, definição adequada de precedência e associatividade de operadores para eliminação completa de conflitos ***shift/reduce***.
Tratamento de Erros e Sincronização	10%	Implementação da recuperação em modo pânico no ***callback p_error(p)***, realizando a sincronização de fluxo através do caractere estável ponto e vírgula (;), permitindo listar múltiplos erros sintáticos sem abortar precocemente.
Relatório Técnico e Organização	10%	Documentação acadêmica detalhada em formato PDF contendo as decisões de ***design***, a ***estrutura da gramática Pascal-S*** e, obrigatoriamente, uma análise técnica aprofundada de eventuais conflitos ***shift/reduce*** obtidos durante o desenvolvimento.

5.1 Entregáveis do Trabalho

Criem grupos de trabalho de até 2 alunos, e para a entrega devem compactar em um único arquivo zip, contendo o nome dos integrantes no formato **TP1_NomeCompleto1_NomeCompleto2.zip**, os seguintes itens obrigatórios:

1. Código-fonte completo e documentado em Python (lex.py, yacc.py e o módulo da Tabela de Símbolos).
2. Apresentação e explicação do código para o professor.
3. Relatório técnico em **PDF** descrevendo em formato acadêmico as decisões adotadas no *design* do compilador e a análise detalhada de conflitos do *parser*.
4. *Scripts* de teste (arquivos de entrada de sucesso, falha léxica e erro semântico de tipo) juntamente com o log de saída correspondente gerado pelo compilador.